Projeto - Baby raiteco

Equipe: Artur Melo, Davi Jesseh, Maria Clara, Gabriel Mapurunga

Conteúdo

O projeto **3**
1 Materiais usados 3
2 Descrição do projeto 3
3 Esquemática 4
4 Bibliotecas 4
4.1 Adafruit SSD1306 4
4.2 RoboEyes 5

Tela OLED **6**
5 Características e especificações 6
6 Conexão I2C 6
7 Transferência de dados 7

Sensores HC-SR04 **8**
8 Características e especificações 8
9 Descrição Geral do Funcionamento 8
10 Configuração dos Pinos 8
11 Medição de Distância 9
12 Leitura Assíncrona dos Sensores 9
13 Triangulação e Determinação da Posição 9

Código **10**

Referências **13**

O projeto

1 Materiais usados

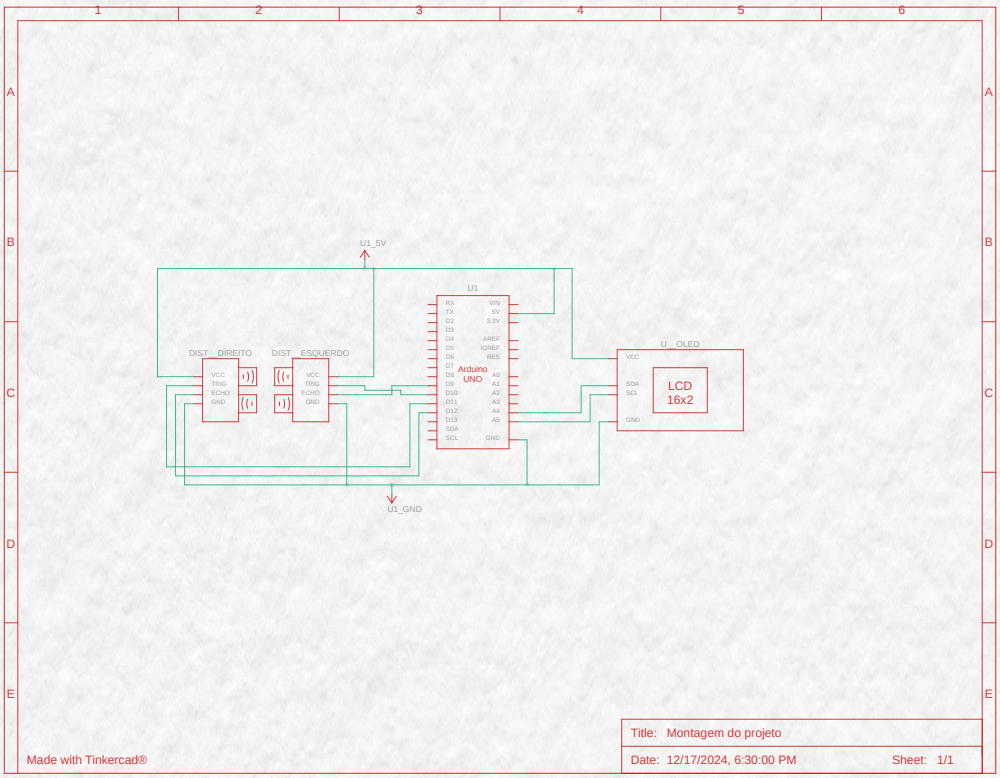
- Arduino Uno
- 2 Protoboads mini
- 2 sensores HC-SR04
- 1 Tela OLED
- Cabos jumper

2 Descrição do projeto

O desafio proposto consistiu no desenvolvimento de um robô interativo com um visor de OLED, que simula dois olhos capazes de seguir o objeto mais próximo. Isso foi feito com dois sensores **HC-SR04** que, por meio de triangulação, eram capazes de identificar onde estava o objeto no eixo X. Além disso, foram desenhados dois olhos na tela OLED com algumas animações simples, fazendo uso da biblioteca **RoboEyes**, com o objetivo de deixar o projeto mais lúdico. A partir disso, os olhos da telinha **OLED** atualizavam sua posição baseando-se na leitura dos sensores, acompanhando o objeto.

Além da funcionalidade técnica, a equipe dedicou atenção ao aspecto visual do projeto, integrando todo o circuito em uma modelagem 3D. Essa abordagem busca deixar o robô mais lúdico e atrativo, especialmente para o público-alvo, que são crianças. Esse projeto visa estimular a curiosidade e o interesse das crianças pela tecnologia e pela robótica.

3 Esquemática



4 Bibliotecas

4.1 Adafruit SSD1306

Essa é a biblioteca principal da telinha OLED, que vai servir como base do funcionamento dela e de integração com outras bibliotecas:

```
#include <Adafruit_SSD1306.h>
```

É importante definir a largura e a altura da tela antes de começar a trabalhar com o componente:

```
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
```

Após isso, é necessário definir os pinos de reset e o endereço da tela (olhar datasheet, mas o da 128x64 é 0x3D)

```
#define OLED_RESET -1  //-1 se for o mesmo pino de reset do Arduino
#define SCREEN_ADDRESS 0x3D
```

Criando o dispositivo:

```
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
```

4.2 RoboEyes

Foi a biblioteca usada para desenhar os olhos e fazer as animações; ela é baseada na Adafruit GFX. É necessário baixá-la para implementar no projeto, instruções no seguinte site: https://github.com/FluxGarage/RoboEyes/tree/main

Para começar importamos a biblioteca e inicializamos o objeto roboEyes

```
#include <FluxGarage_RoboEyes.h>
roboEyes roboEyes;
```

Após isso, precisamos definir a largura e a altura da tela, além do número de frames da animação:

```
roboEyes.begin(SCREEN_WIDTH, SCREEN_HEIGHT, 100);
```

Então definimos as configurações básicas dos olhos, podendo ser alteradas para fazer outros formatos:

```
roboEyes.setWidth(36, 36); // byte olho esquerdo, byte olho direito
roboEyes.setHeight(36, 36); // byte olho esquerdo, byte olho direito
roboEyes.setBorderradius(8, 8); // byte olho esquerdo, byte olho direito
roboEyes.setSpacebetween(10); // int espaço -> pode ser negativo
```

Tela OLED

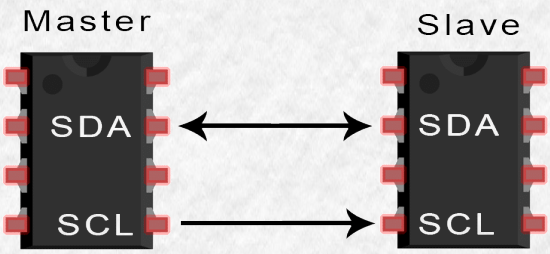
5 Características e especificações

Tecnologia do Display	OLED (LED orgânico)
Interface MCU	I2C
Tamanho da tela	0.96 polegadas
Resolução	128x64
Voltagem operacional	3.3V - 5V
Corrente operacional	20 mA
Ângulo max. de visibilidade	160°
Caracteres por linha	21
Numero de linhas de caracteres	7

6 Conexão I2C

I2C ou **Inter Integrated Circuit** é um protocolo de comunicação que permite que um **master** controle vários **slaves**, isso faz com que, por exemplo, seja possível controlar vários LEDs com um só dispositivo ou comando.

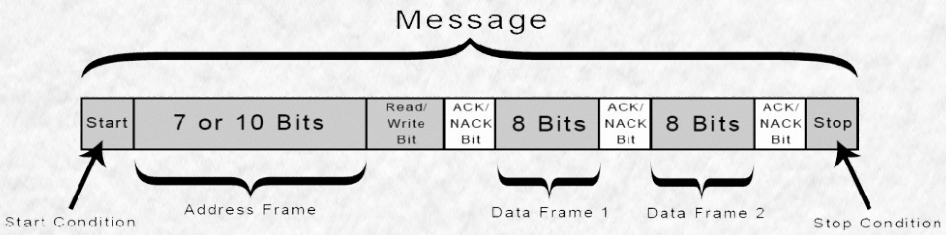
Normalmente os componentes que usam esse tipo de comunicação apresentam dois pinos chamados **SDA(Dados)** e **SCL(Clock)**, funcionando da seguinte maneira:



Então o SDA serve para a comunicação entre **master** e **slave**, ou seja, para mandar e receber dados, e como o I2C é um sistema de comunicação serial, os dados são man-

dados bit por bit pelo fio(SDA). Já o SCL serve para o **clock**, que controla a velocidade à qual os dados são mandados/recebidos, indo de 100Kb/s até 5Mb/s.

7 Transferência de dados



Os dados são transferidos em formato de **mensagem**, a qual tem 7 a 10 bits, mas o formato da mensagem não importa para o nosso uso. Basicamente, o master vai mandar um pacote de dados para os slaves, esse pacote de dados vai estar com o "nome"(endereço) do slave que é para receber a mensagem, quando os nomes batem a transmissão/recepção de dados vai estar aberta. Quando a comunicação termina, quem mandou os dados recebe um sinal de confirmação(bits ACK/NACK). Então, um master pode se conectar a 2^7 ou a 2^{10} slaves.

Sensores HC-SR04

8 Características e especificações

Alimentação	5V
Corrente operacional	15mA
Ângulo de efeito	até 15°
Alcance	2m - 4m
Precisão máxima	3mm/cm
Largura de pulso trigger	10μs

9 Descrição Geral do Funcionamento

O código utiliza dois sensores HC-SR04, configurados como sensores esquerdo e direito. Cada sensor mede a distância até um objeto em frente ao sistema. Com base nas leituras de ambos os sensores, a posição relativa do objeto (coordenada x) é calculada usando triangulação.

10 Configuração dos Pinos

Os sensores estão conectados aos seguintes pinos do microcontrolador:

- **Sensor Esquerdo:**
  - Trigger: Pino 9
  - Echo: Pino 10
- **Sensor Direito:**
  - Trigger: Pino 11
  - Echo: Pino 12

11 Medição de Distância

A função `getDistance()` realiza a medição da distância para um sensor específico. O processo é descrito abaixo:

1. O pino `Trigger` é configurado como `LOW` por 2 μs para garantir que o sensor esteja em estado inicial.
2. Um pulso de 10 μs é enviado ao pino `Trigger` para iniciar a medição.
3. O sensor emite um pulso ultrassônico e o pino `Echo` recebe o sinal refletido do objeto.
4. O tempo de duração do pulso no pino `Echo` é medido utilizando a função `pulseIn()`.
5. A distância é calculada com a fórmula:

$$\text{distancia} = \frac{\text{tempo} \times 0.0343}{2}$$

   Onde o fator 0.0343 converte o tempo em centímetros (com base na velocidade do som).

Se a distância calculada estiver fora do intervalo aceitável (1 cm a 100 cm), a função retorna -1 indicando uma leitura inválida.

12 Leitura Assíncrona dos Sensores

Para evitar atrasos no programa, as leituras dos sensores são realizadas de forma assíncrona com base no tempo atual (obtido com `millis()`). Cada sensor é lido em intervalos de 100 ms:

- O tempo atual é comparado com o último tempo de leitura para cada sensor.
- Se o intervalo de 100 ms foi atingido, a distância do respectivo sensor é atualizada chamando `getDistance()`.

13 Triangulação e Determinação da Posição

Com as distâncias d_1 (sensor esquerdo) e d_2 (sensor direito) calculadas, a posição x do objeto é determinada pela fórmula:

$$x = \frac{d_1^2 - d_2^2}{2L}$$

Onde L é a distância fixa entre os dois sensores, definida como 12 cm no código.

Código

```
#include <Adafruit_SSD1306.h>

// Pinos dos sensores HC-SR04
#define trigPinLeft 9    // Trigger do Sensor Esquerdo
#define echoPinLeft 10   // Echo do Sensor Esquerdo
#define trigPinRight 11  // Trigger do Sensor Direito
#define echoPinRight 12  // Echo do Sensor Direito
#define MAX_DISTANCE 100 // Distância máxima em cm

unsigned long tempoAnterior1 = 0;  // Armazena o tempo anterior para o sensor 1
unsigned long tempoAnterior2 = 0;  // Armazena o tempo anterior para o sensor 2
const long intervalo = 100;          // Intervalo entre leituras (em milissegundos)


// Variáveis para triangulação
const float L = 12.0; // Distância entre os sensores em cm
float x = 0;          // Posição X calculada do objeto

float d1 = 0.0;
float d2 = 0.0;

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

#define OLED_RESET -1  //-1 se for o mesmo pin de reset do arduino
#define SCREEN_ADDRESS 0x3D //Endereço da tela

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);


// Criação do objeto OLED
#include <FluxGarage_RoboEyes.h>
roboEyes roboEyes;
```

```
void setup() {
  Serial.begin(9600);
  // Startup OLED Display
  // SSD1306_SWITCHCAPVCC = generate display voltage from 3.3V internally
  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) { // Address 0x3C or 0x3D
    Serial.println(F("SSD1306 allocation failed"));
    for(;;); // Don't proceed, loop forever

  pinMode(trigPinLeft, OUTPUT);
  pinMode(echoPinLeft, INPUT);
  pinMode(trigPinRight, OUTPUT);
  pinMode(echoPinRight, INPUT);
  }
//Inicializando o RoboEyes
roboEyes.begin(SCREEN_WIDTH, SCREEN_HEIGHT, 100); // Largura, Altura, Framerate

roboEyes.setAutoblinker(ON, 8, 3); // Animação de piscar -> bool ativo, int interval

roboEyes.setWidth(36, 36); // byte olho esquerdo, byte olho direito
roboEyes.setHeight(36, 36); // byte olho esquerdo, byte olho direito
roboEyes.setBorderradius(14, 14); // byte olho esquerdo, byte olho direito
roboEyes.setSpacebetween(10); // int espaço -> pode ser negativo


}
void loop() {
  unsigned long tempoAtual = millis();
  // Obter distâncias dos sensores

  // Sensor Esquerdo - Leia sem atraso
  if (tempoAtual - tempoAnterior1 >= intervalo) {
    d1 = getDistance(trigPinLeft, echoPinLeft);
    tempoAnterior1 = tempoAtual;
  }
  // Sensor Direito - Leia sem atraso
  if (tempoAtual - tempoAnterior2 >= intervalo) {
    d2 = getDistance(trigPinRight, echoPinRight);
    tempoAnterior2 = tempoAtual;
  }

  // Evitar leituras inválidas
  if (d1 == -1){d1 = MAX_DISTANCE;}
  if (d2 == -1){d2 = MAX_DISTANCE;}
```

```
  //Calcular posição X usando triangulação
  x = (pow(d1, 2) - pow(d2, 2)) / (2 * L);

  determinarEstado(x);
  roboEyes.update();
}

long getDistance(int trigPin, int echoPin) {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long duration = pulseIn(echoPin, HIGH, 20000UL);

  float distancia = (duration / 2.0) * 0.0343; //Converte para cm
  if (distancia >= MAX_DISTANCE || distancia <= 0.1) {
    return -1; // Valor fora do alcance
  }
  else{
    return distancia;
    }
}

void determinarEstado(float x) {
  if (x < -2.0) { //Esquerda
      roboEyes.setPosition(W);
      roboEyes.setCuriosity(ON);
      roboEyes.setMood(DEFAULT);
  }

  else if (x >= -2.0 && x <= 2.0) { // Centro
      roboEyes.setPosition(DEFAULT);
      roboEyes.setCuriosity(OFF);
      roboEyes.setMood(HAPPY);
  }

  else { //Direita
      roboEyes.setPosition(E);
      roboEyes.setCuriosity(ON);
      roboEyes.setMood(DEFAULT);
  }
}
```

Referências

Adafruit. Adafruit SSD1306 Library. Disponível em: https://github.com/adafruit/Adafruit_SSD1306/tree/master. Acesso em: 13 dez. 2024.

CIRCUIT BASICS. Basics of the I2C Communication Protocol. Disponível em: https://www.circuitbasics.com/basics-of-the-i2c-communication-protocol/. Acesso em: 13 dez. 2024.

MORGAN, Elijah J. HC-SR04 Ultrasonic Sensor. 2014. Disponível em: https://www.alldatasheet.com/datasheet-pdf/pdf/1132204/ETC2/HCSR04.html. Acesso em: 19 dez. 2024.

FluxGarage. RoboEyes Project. Disponível em: https://github.com/FluxGarage/RoboEyes/tree/main. Acesso em: 14 dez. 2024.